

TITLE: EXP NO.: 4 QUERYING LEDGER DATA WITH HYPERLEDGER FABRIC

1. AIM

To query and retrieve ledger data from a Hyperledger Fabric blockchain network using JavaScript chaincode, and verify asset information stored in the world state database.

2. TOOLS / SOFTWARE REQUIRED

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

3. HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

4. PROCEDURE

Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC  
-ccl javascript  
-ccn basic  
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 4: Initialize the Ledger

Load the ledger with sample assets.

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
--cafile $ORDERER_CA  
-C mychannel  
-n basic  
-c '{"function":"InitLedger","Args":[]}'
```

Step 5: Query All Assets

Retrieve all asset records stored in the blockchain ledger.

```
peer chaincode query  
-C mychannel  
-n basic  
-c '{"Args":["GetAllAssets"]}'
```

Step 6: Query a Specific Asset

Retrieve the details of asset1.

```
peer chaincode query  
-C mychannel  
-n basic  
-c '{"Args":["ReadAsset","asset1"]}'
```

Step 7: Create a New Asset

```
peer chaincode invoke  
-o localhost:7050  
--ordererTLSHostnameOverride orderer.example.com  
--tls  
--cafile $ORDERER_CA  
-C mychannel  
-n basic  
-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'
```

Step 8: Query the Newly Created Asset

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["ReadAsset","asset20"]}'
```

Step 9: Query All Assets Again

Retrieve all assets to verify that the new asset has been added.

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["GetAllAssets"]}'
```

Step 10: Query a Non-Existing Asset

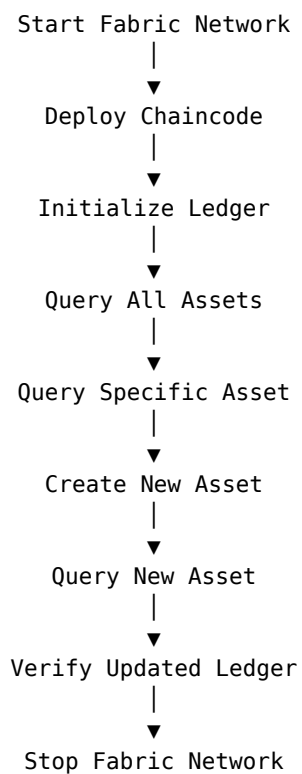
Attempt to retrieve an asset that does not exist.

```
peer chaincode query
-C mychannel
-n basic
-c '{"Args":["ReadAsset","asset50"]}'
```

Step 11: Stop the Fabric Network

```
./network.sh down
```

5. FLOW OF PROCESS



6. SUMMARY OF KEY COMMANDS

Purpose	Command
Navigate	<code>cd ~/fabric-dev/fabric-samples/test-network</code>
Start network	<code>./network.sh up createChannel -ca</code>
Deploy chaincode	<code>./network.sh deployCC</code>
Invoke transaction	<code>peer chaincode invoke</code>
Query ledger	<code>peer chaincode query</code>
Stop network	<code>./network.sh down</code>

7. OUTPUT

The terminal output below is rendered from the output blocks supplied in the source experiment.

Output 1

Experiment 4 - Fabric Output

```
$ Fabric Network Started Successfully
Chaincode Installed Successfully
Ledger Initialized Successfully
All Assets Retrieved Successfully
Specific Asset Retrieved Successfully
New Asset Created Successfully
Ledger Updated Successfully
Query Completed Successfully
```

Fig. 4.1 – Fabric output corresponding to the source experiment output.

8. RESULT

The ledger data stored in the Hyperledger Fabric blockchain network was successfully queried using JavaScript chaincode. Asset records were retrieved from the world state database, specific asset details were verified, and newly created assets were successfully queried, demonstrating efficient data retrieval and verification in a permissioned blockchain environment. Exp No.: 5 Integration with External Systems using Hyperledger Fabric